

Generative AI technical Questions

1. Explain the difference between generative and discriminative models. Give one example of each.

The difference between generative models and discriminative models is that discriminative models learn the boundary between two classes or more to differentiate between them, as they learn $P(\text{class} | \text{features})$. So it actually focuses on using the features to differentiate between different things, while generative models learn the underlying distribution of the data, so they can actually generate new samples from what they have learned. They can learn $P(\text{features} | \text{class})$ and $P(\text{class})$, so they learn how the features are distributed for each class. Examples of discriminative models include logistic regression and other classification models, while examples of generative models include ChatGPT, which is an LLM that can generate new text based on the patterns it learned from text, and image generation models like Nano Banana that can create new images.

2. What is the purpose of transformers in modern AI? How does it work briefly?

The purpose of transformers actually came to solve a problem from the technology that came before it, which was the RNNs, which struggled with long-term dependencies and could not process a lot of data at once due to handling the sequence one word at a time, as they lacked the attention-based logic that transformers have. Transformers can process the words in a sentence in parallel, treating them as vectors, and calculating how much each word relates to the other important words around it, giving them different weights, which is known as the attention process. They can also use GPUs efficiently due to their parallel processing way of working, and that's why ChatGPT, Gemini, and these LLMs were able to be built and scaled using transformers, as they allowed the model to capture relationships between words that are far apart in a sequence, which was much harder for RNNs to handle.

3. Describe the difference between supervised, unsupervised, and reinforcement learning.

Supervised learning must have the label during the learning process, as it tries to learn the weights by comparing the prediction to the labels and adjusting the weights based on that using the loss function and gradient descent algorithm, for example. Some of the most famous supervised learning algorithms are logistic regression and regression models in general. Unsupervised learning tries to cluster the data based on the relation between them, as it tries to learn the patterns of the data and come out with a rule based on that because

we don't have a label in this case. Some of the most famous algorithms for unsupervised learning are K-Means and PCA. Reinforcement learning tries to learn using the principle of reward and punishment, as it keeps repeating a process and when it achieves what is wanted, it gives a reward and enhances this action, and when it does something we don't want, it penalizes it, making this other action less likely to be chosen. Some famous reinforcement learning algorithms are Q-Learning, SARSA, and Deep Q-Networks (DQN).

4. What are tokens in the context of Large Language Models? Why are they important?

Tokens are the units that the model reads and processes, and the number of tokens also measures how much text you are sending to or receiving from the model. The model understands tokens as pieces of text, which can sometimes be complete English words like "the", "is", "she", and "it", but they can also be parts of words. There is a rough rule that 1 token is around 0.75 of an English word, but this is not a fixed rule. For harder or less common words, the model can divide them into separate smaller parts to analyze them, which can consume more tokens. This is also why some non-English languages can consume more tokens than English for the same amount of text, depending on the tokenizer, because their words can be separated into many smaller pieces and each one consumes a token.

The idea of tokens came from making a way to treat the sentences given to the model, as we can't make a dictionary to memorize all the words in the world because this would be very inefficient. So tokenization came to solve this by dividing text into smaller pieces that the model can understand and work with. Also, if you write something wrong like having a typo, the tokenizer might divide your word into separate or unusual pieces because it doesn't match the common patterns it has seen, and this can sometimes consume more tokens. That's why typos can sometimes increase the number of tokens used.

5. Explain the concept of attention in neural networks. How does it improve model performance?

The attention, as we explained, works with the Query, Key, Value principle. Instead of treating each single word in the sentence equally, it calculates how important each word is compared to the other words around it, and it can do this all at once which is the parallel processing idea. It can actually compare the Query to the Keys around the word, like how the YouTube search engine works when you search for something, as it compares your search query to the video titles or information about the videos (keys), and then the values

are the actual information that the model uses from the relevant videos. This is also how attention works here by doing a dot product between the Query and the Keys to calculate how much they relate to each other, then using these scores as attention weights to combine the Values. This gives higher attention to the words that are more relevant to the word being queried, even if they are far away in the sentence, so the model can understand the context better. That's why when having the same word in the sentence with different meanings, the LLM would not find any problem and would be able to understand the meaning of the word based on the context of the sentence.

6. What is prompt engineering, and why is it crucial for Gen AI applications? State the different types of prompts used.

Prompt engineering is a way to get the best results from a model without modifying the model itself, so there are a lot of ways for prompt engineering. Zero-shot method is when you ask for what you want directly without giving any examples, one-shot is when you give one example of what you want, and few-shot is when you lay some examples and show how the output should be. Chain-of-thought is when you encourage the model to solve a problem step by step instead of just giving the final answer. There is also role prompting, where you give the model a specific role like asking it to act as a teacher or a software engineer, and context-based prompting where you give the model relevant information that it should use when generating the answer. Prompt engineering is crucial for Gen AI applications because the same model can give very different results depending on how the prompt is written, so a good prompt can make the output more accurate, relevant, consistent, and closer to what the user actually wants.

7. Explain how RAG (Retrieval-Augmented Generation) works.

RAG, which stands for Retrieval-Augmented Generation, is a way to make the LLM use external knowledge when generating its answer instead of depending only on the knowledge it learned during training. The idea is that we have some documents or a knowledge base, and we first divide the documents into smaller chunks and convert these chunks into embeddings, which are stored in a vector database. When the user asks a question, the question is also converted into an embedding, and the system searches the vector database to retrieve the chunks that are most relevant to the question. These retrieved chunks are then added to the prompt and given to the LLM as context, and the LLM uses this context to generate the final answer. This makes the model able to answer questions using specific or updated information that was not necessarily part of its original

training data, and it can also reduce hallucinations because the model has relevant information to base its answer on.

8. What is embedding in AI, and why is it important for retrieval tasks?

Embedding is important for retrieval because it is the core of RAGs and how they work, as we take the prompt and embed it to represent its meaning as a vector, and then compare this vector to the other vectors of the knowledge source to get the most similar vectors in our vector database. This allows us to retrieve the most relevant content that can be given along with the question to the LLM to generate the correct answer, and this is the augmented part of RAG.

9. How does temperature affect the output of a language generation model?

Temperature affects the output of a language generation model by controlling how random or predictable the model's answers are. The model gives probabilities to the possible next tokens, and the temperature changes how these probabilities are distributed. When we use a low temperature, the model becomes more deterministic and tends to choose the tokens with the highest probability, so the answers become more consistent and predictable. When we increase the temperature, the probabilities become more spread out, so the model has a higher chance of choosing less probable tokens, which makes the output more diverse, creative, and random. So basically, low temperature is better when we want more consistent and predictable answers, while higher temperature can be useful when we want more creativity and different possible outputs.

10. What is chain-of-thought prompting, and why can it improve reasoning in LLMs?

Chain-of-thought prompting is a way of prompting the LLM to solve a problem step by step instead of directly giving the final answer. The idea is that when the problem is complex, the model can break it down into smaller steps and solve each part one by one, which helps it follow the logic better and reach a more accurate final answer. It is especially useful for tasks like mathematical problems, logical reasoning, and problems that have multiple steps. So basically, instead of asking the model to jump directly to the answer, we guide it to reason through the problem step by step.

11. What is the difference between RAG and fine-tuning? Give a use case for each.

The main difference between RAG and fine-tuning is that RAG gives the LLM external knowledge without changing the model itself, while fine-tuning actually changes the model by training it on a specific dataset.

In RAG, we have some external knowledge source like company documents, and when the user asks a question, we retrieve the most relevant information from the knowledge base and give it to the LLM along with the question. For example, a company can use RAG to build a chatbot that answers employees' questions based on the company's internal documents and policies.

For fine-tuning, we train the model on a specific dataset so it learns a specific style, behavior, or task better. For example, we can fine-tune a language model on customer support conversations so it learns how to respond in a specific customer service style.

So basically, RAG gives the model new information at the time of answering, while fine-tuning changes the model itself through additional training.

12. Explain the possible improvements that can be applied to a Naive RAG (Retrieval-Augmented Generation) system, both pre-retrieval and post-retrieval.

A Naive RAG system can be improved in two main stages, which are pre-retrieval and post-retrieval.

For **pre-retrieval**, the goal is to improve the quality of the data before we actually search for the relevant information. We can improve the document cleaning and chunking, as bad chunking can make the retrieved information incomplete or unrelated. We can also choose a better embedding model to represent the meaning of the documents and the user query better. We can also improve the query itself by rewriting or expanding it so the retrieval system can understand what the user is actually asking for.

For **post-retrieval**, the goal is to improve the results that were retrieved before giving them to the LLM. We can use a **reranking model** to reorder the retrieved chunks and put the most relevant ones first. We can also remove irrelevant or duplicate chunks and only send the most useful information to the LLM. Another improvement is to use the retrieved information as context in a better way so the LLM can generate an answer based on the relevant information instead of irrelevant retrieved data.

So basically, pre-retrieval improves the data, chunks, embeddings, and query before searching, while post-retrieval improves and filters the results after searching and before giving them to the LLM.

**13. What factors should you consider when choosing an LLM for an application?
Consider accuracy, latency, cost, context window, privacy, and reasoning ability.**

When choosing an LLM for an application, we should consider several factors depending on what the application needs. Accuracy is important to make sure the model gives reliable and useful answers. Latency is important when we need fast responses, especially in real-time applications. Cost is also important because some models are much more expensive to use than others.

We should also consider the context window, especially if the application needs to handle long documents or large amounts of information. Privacy is important when dealing with sensitive or private data, so we need to know how the provider handles the data. Finally, we should consider the model's reasoning ability, especially for complex tasks that require multiple steps.

So basically, we choose the LLM based on the balance between accuracy, speed, cost, context size, privacy, and reasoning ability depending on the application's requirements.

14. How can you evaluate whether an LLM-generated answer is grounded in the provided documents?

We can evaluate if an LLM-generated answer is grounded by checking whether the information in the answer can actually be supported by the provided documents. We can compare the answer with the retrieved chunks and check if the claims made by the LLM are present or logically supported by them.

We can also use an evaluator or another LLM to check each claim against the retrieved context and identify unsupported information or hallucinations.

So basically, we check if the answer is supported by the retrieved documents and does not contain information that came from nowhere.

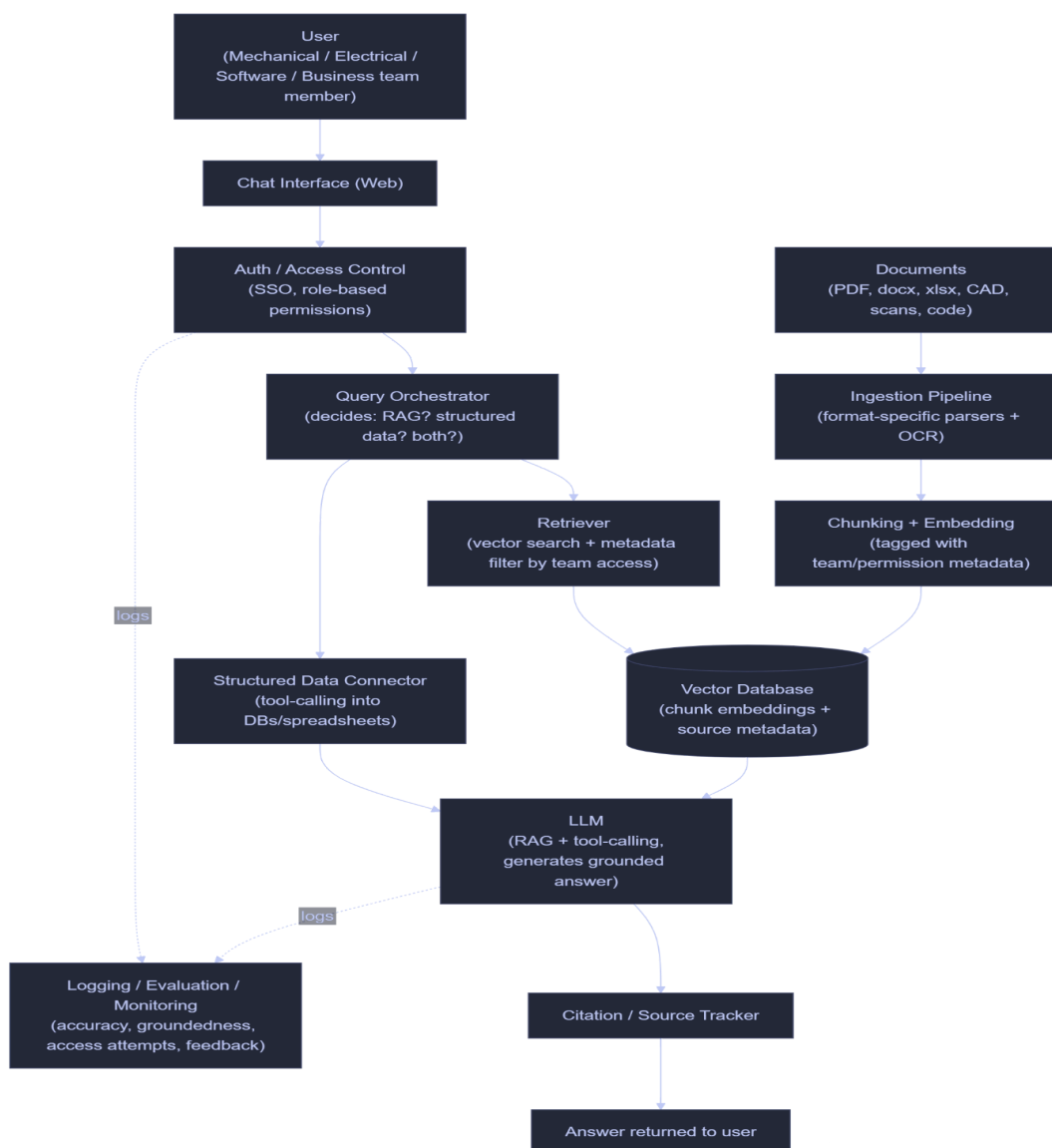
15. How do you think computer vision could be integrated with Generative AI ?

Computer vision can be integrated with Generative AI by allowing the model to understand images and videos and then use this information to generate useful outputs. For example, a vision model can analyze an image and extract important information from it, and then a generative model can use this information to answer questions, describe the image, or generate a response based on what it sees.

It can also be used in applications like medical imaging, where computer vision can analyze an X-ray or MRI and Generative AI can explain the findings in a more understandable way. So basically, computer vision helps the AI understand visual data, while Generative AI can use this understanding to generate text, images, or other useful outputs.

16. Design an AI assistant for CURT.

a. Draw the system architecture and explain the purpose of each component.



a. System Architecture

User → Chat Interface → Authentication → Query Orchestrator

Then the system chooses:

- **Documents** → Retriever → Vector Database
- **Structured Data** → Database Connector

Then:

LLM → Citation → Answer

Components

- **Authentication:** Checks user permissions.
- **Query Orchestrator:** Decides where to search.
- **Ingestion:** Processes different file types.
- **Vector Database:** Stores document embeddings.
- **Retriever:** Finds relevant information.
- **LLM:** Generates the answer.
- **Citation:** Shows the sources.

b. Main Challenges

1. **Different file formats** → Difficult to process.
2. **Bad data** → Missing, noisy, or inconsistent information.
3. **Large documents** → Context/token limits.
4. **Retrieval** → May find irrelevant information.
5. **LLM accuracy** → May hallucinate or give unsupported answers.

c. Solutions

1. **Use suitable parsers and OCR** for different file types.
2. **Clean and validate the data.**
3. **Use smart chunking** for large documents.
4. **Use hybrid search** (keyword + vector search).
5. **Use citations and grounded RAG** to reduce hallucinations.
6. **Use access control** to prevent unauthorized access.

d. Evaluation

Test the system using questions with known correct answers.

Check:

- **Retrieval accuracy** → Did it find the right information?
- **Answer accuracy** → Is the answer correct?
- **Groundedness** → Is the answer supported by sources?
- **Citation accuracy** → Are the sources correct?
- **Security** → Can users access only their allowed data?
- **User feedback** → Are the answers useful?